



KARADENİZ TEKNİK ÜNİVERSİTESİ

FEN FAKÜLTESİ

BİLGİSAYAR BİLİMLERİ BÖLÜMÜ

VERİTABANI YÖNETİMİ

PROJE RAPORU

Sağlık Ocağı Yönetim Sistemi (SOYS)

Öğrencinin Adı ve Soyadı : Melih KAYADUVAR

Öğrenci Numarası : 439308

Ders Sorumlusu : Dr. Öğr. Üyesi Uğur ŞEVİK

Proje Kaynak Kodları : [GitHub Repository](#)

İçindekiler

Giriş.....	4
Kullanılan Teknolojiler	5
Gereksinimler.....	5
Varlıklar.....	6
Kavramsal Model.....	7
Mantıksal Model	7
Fiziksel Model	8
Ekran Görüntüleri	12
a)Yönetici Arayüzü.....	12
1. Doktor Listesi:.....	13
2. Doktor Ekleme	13
3. Doktor Silme	14
4. Hasta Listesi	14
5. Hasta Ekleme.....	15
6. Hasta Düzenleme.....	16
7. Hasta Silme	17
8. İlaç Yönetimi.....	18
9. Tetkik Tanımlama.....	19
b) Doktor Arayüzü.....	20
1. Hasta Listesi	20
2. Ziyaret	21

3. Ziyaret	22
4. Reçete Listesi	23
5. Reçete Kalemleri:	23
6. Reçete Oluştur	24
7. Reçete Silme.....	25
8. İstenen Tetkik Listesi.....	25
9. İstenen Tetkik Düzenleme	26
10. İstenen Tetkik Silme	27
11. Tetkik İste	27

Giriş

Bu proje, bir sağlık merkezinin (sağlık ocağı) tüm idari ve klinik süreçlerini uçtan uca yönetebilmek amacıyla geliştirilmiş kapsamlı bir **Sağlık Ocağı Yönetim Sistemi (SOYS)** otomasyonudur. Günümüz sağlık hizmetlerinde verinin hızlı işlenmesi, hasta geçmişine saniyeler içinde ulaşılması ve tıbbi kayıtların hata payı olmaksızın saklanması hayati önem taşımaktadır. Bu çalışma, geleneksel kayıt yöntemlerinin yerini alacak, doktor ve yönetici iş yükünü minimize eden dijital bir çözüm sunmaktadır.

Projenin temel odak noktaları ve kapsamı şu maddeler altında toplanmaktadır:

- **Yetkilendirme ve Güvenlik:** Sistem, çoklu kullanıcı desteği ile tasarlanmıştır. Doktorlar ve sistem yöneticileri (admin) kendilerine özel arayüzler ve yetki setleri ile sisteme dahil olurlar. Bu sayede hassas hasta verilerinin güvenliği ve mahremiyeti korunmaktadır.
- **Merkezi Hasta Bilgi Sistemi:** Hastaların sadece kimlik bilgileri değil; alerjileri, kronik hastalıkları ve kan grupları gibi kritik tıbbi verileri merkezi bir veritabanında saklanır. Bu yapı, doktorun hastayı muayene etmeden önce risk faktörlerini bir bakışta görmesini sağlar.
- **Klinik Süreç Yönetimi:** Muayene (ziyaret) süreci; şikayet, bulgu, tanı ve tedavi notları şeklinde dijital formlara dökülmüştür. Her muayene, ilgili doktor ve hasta ile ilişkili bir şekilde tarih-saat damgasıyla kayıt altına alınarak geriye dönük tıbbi arşiv oluşturulur.
- **İlaç ve Reçete Otomasyonu:** Sistem içerisinde tanımlı olan geniş bir ilaç envanteri üzerinden reçete yazımı gerçekleştirilir. Bu modül, yanlış ilaç yazımını azaltmayı ve hastanın kullandığı ilaçların takibini kolaylaştırmayı amaçlar.
- **İstatistiksel Karar Destek:** Yönetici paneli aracılığıyla sunulan istatistiksel veriler (toplam hasta sayısı, muayene yoğunluğu, en sık kullanılan ilaçlar vb.), kurumun kaynaklarını daha verimli yönetmesine olanak tanıyan bir karar destek mekanizması işlevi görür.

Sonuç olarak bu otomasyon; nesne yönelimli programlama (OOP) prensipleri kullanılarak, esnek bir veritabanı mimarisi ve kullanıcı dostu bir grafik arayüz (GUI) ile entegre edilmiştir. Sistem, modern sağlık standartlarına uygun olarak veri bütünlüğünü (Data Integrity) ve erişilebilirliği en üst seviyeye çıkarmayı hedeflemektedir.

Kullanılan Teknolojiler

Sistemin geliştirilmesinde performans, görsel yönetim ve güçlü veritabanı altyapısı kriterleri göz önünde bulundurularak aşağıdaki teknolojiler kullanılmıştır:

- **C++ (C++17 Standartı):** Uygulamanın temel iş mantığı ve nesne yönelimli yapısı bu dil ile geliştirilmiştir. Sınıf yapıları (OOP) ve bellek yönetimi avantajları nedeniyle tercih edilmiştir.
- **Qt Framework & Designer:** Kullanıcı arayüzlerinin (GUI) tasarımı ve uygulama içi etkileşimlerin yönetimi için kullanılmıştır. Qt'nin sunduğu sinyal-slot mekanizması ile kullanıcı işlemleri kod tarafına entegre edilmiştir.
- **SSMS (SQL Server Management Studio):** Veritabanı tasarımı, tablo ilişkilerinin kurulması ve SQL sorgularının optimize edilmesi süreçlerinde temel yönetim aracı olarak kullanılmıştır.
- **SQL (Relational Database):** Verilerin tutarlılığı ve ilişkisel olarak (Hasta-Doktor-Ziyaret) saklanması amacıyla standart SQL yapısı tercih edilmiştir. Uygulama, SQL Server altyapısı ile Qt SQL modülü üzerinden haberleşmektedir.
- **CMake:** Projenin bağımlılıklarını yönetmek ve derleme süreçlerini otomatize etmek için derleme sistemi olarak kullanılmıştır.

Gereksinimler

Sistemin işleyişi ve kullanıcı ihtiyaçları doğrultusunda belirlenen temel gereksinimler aşağıda maddeler halinde sunulmuştur:

- **Yetkilendirme:** Sisteme giriş yapacak olan kullanıcılar (Doktor ve Yönetici) kendi kimlik numaraları ve şifreleri ile doğrulanmalıdır.
- **Hasta Kayıt ve Takibi:** Yeni gelen hastaların kişisel bilgileri, iletişim detayları ve kan grubu gibi kritik verileri sisteme kaydedilmeli; kayıtlı hastaların bilgileri güncellenebilmelidir.
- **Tıbbi Geçmiş Yönetimi:** Hastaların varsa alerji ve kronik hastalık bilgileri sistemde tutulmalı, muayene esnasında doktorun erişimine sunulmalıdır.
- **Muayene (Ziyaret) İşlemleri:** Her muayene için şikayet, bulgu ve tanı girişleri yapılabilmesi; bu kayıtlar ilgili hasta ve doktor ile ilişkilendirilerek tarih bazlı saklanmalıdır.
- **Reçete ve İlaç Envanteri:** Sistemde kayıtlı olan ilaçlar arasından seçim yapılarak hastaya reçete oluşturulabilmelidir.
- **İstatistiksel Raporlama:** Yönetici paneli üzerinde sistemdeki toplam hasta, doktor, ziyaret ve ilaç sayıları gibi genel veriler anlık olarak izlenebilmelidir.
- **Veri Güvenliği:** Veritabanındaki tüm tıbbi ve kişisel kayıtlar, ilişkisel veritabanı kurallarına uygun olarak tutarsızlıktan uzak ve güvenli bir şekilde depolanmalıdır.

Varlıklar

a) **Hasta:**

Sistemdeki temel öznedir. TC Kimlik No, ad, soyad, telefon, doğum tarihi, cinsiyet, kan grubu, adres bilgilerini barındırır. Ayrıca hastaya ait **alerjiler** ve **kronik hastalıklar** ayrı tablolarla bu varlığa bağlı olarak takip edilir.

b) **Doktor:**

Sağlık ocağında görev yapan personeli temsil eder. Ad, soyad, telefon, diploma numarası ve uzmanlık alanı bilgilerinden oluşur. Sisteme girişi için kullanılan şifre bilgileri ile ilişkilidir.

c) **Ziyaret (Muayene):**

Hasta ve doktor arasındaki etkileşimi temsil eden merkezi varlıktır. Tarih/saat, hastanın şikayeti, doktorun koyduğu tanı ve tedavi notlarını içerir. Bu varlığa bağlı olarak hastanın o anki **bulguları** (açıklama ve tip bazlı) kaydedilebilir.

d) **İlaç:**

Sistemde tanımlı olan tıbbi materyallerdir. İlacın barkod numarası, adı, etken maddesi, formu ve standart dozaj bilgilerini kapsar.

e) **Reçete:**

Belirli bir muayene (ziyaret) sonucunda oluşturulan üst belgedir. Reçetenin oluşturulma tarihi ve geçerlilik süresi bilgilerini barındırır.

f) **Reçete Kalemi:**

Reçete ile İlaç arasındaki bağlantıyı kuran detay tablosudur. Reçetede her bir ilacın kullanım şekli, dozu, periyodu ve kaç adet (miktar) verildiği bilgisini tutar.

g) **Tetkik:**

Sistemde yapılabilecek laboratuvar veya görüntüleme işlemlerinin tanımlandığı katalogdur. Tetkik adı, açıklaması ve kadın/erkek için normal referans değerlerini (nderkek, ndkadin) barındırır.

h) **İstenen Tetkikler:**

Muayene sırasında doktorun talep ettiği somut işlemlerdir. İlgili tetkikin istem tarihi, sonuç tarihi, sonuç verisi, doktor yorumu ve işlemin güncel durumunu barındırır

i) **Bulgular:**

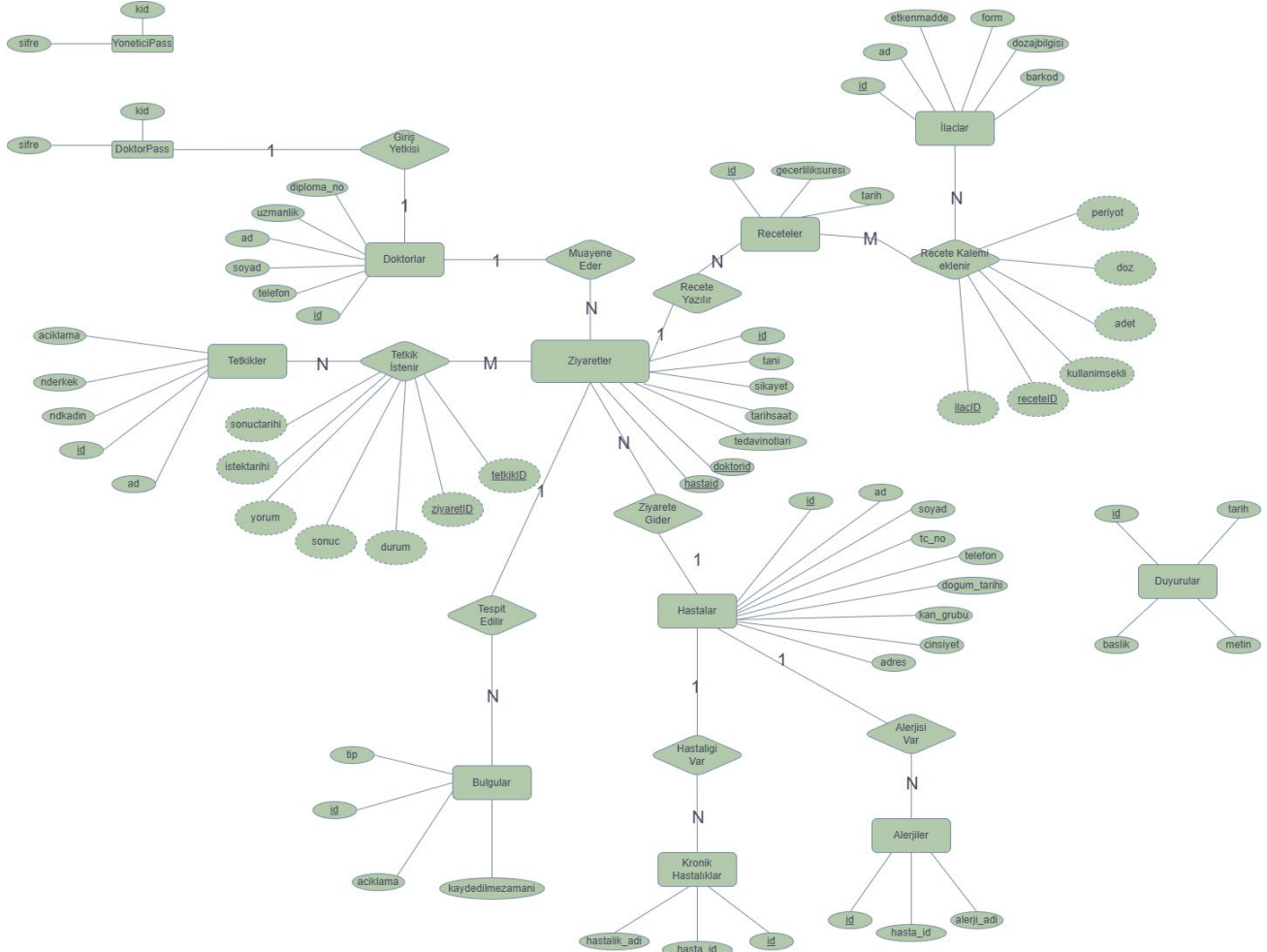
Muayene esnasında hastada gözlemlenen spesifik tıbbi bulguların açıklama ve tip olarak kaydedilmesini sağlar.

j) **Duyurular:**

Sistem üzerinden yayınlanan genel bilgilendirme başlıklarını, metinlerini ve tarihlerini içerir.

Kavramsal Model

Kavramsal model, Sağlık Ocağı Sistemi'nin veritabanı altyapısını oluşturan varlıkların birbirleriyle olan ilişkilerini ve sistemin genel veri akışını soyut bir çerçevede tanımlar.



Mantıksal Model

- **doktorlar** (id, ad, soyad, telefon, diploma_no, uzmanlik)
- **hastalar** (id, tc_no, ad, soyad, telefon, dogum_tarihi, kan_grubu, cinsiyet, adres)
- **hasta_alerjiler** (id, hasta_id (hastalar tablosuna referans verir), alerji_adi)
- **hasta_kronik_hastaliklar** (id, hasta_id (hastalar tablosuna referans verir), hastalik_adi)
- **ilaclar** (id, barkod, ad, etkenmadde, form, dozajbilgisi)
- **tetkikler** (id, ad, aciklama, nderkek, ndkadin)

- **tetkikIstenir** (id, ziyaretID, tetkikID, istektarihi, sonuctarihi, sonuc, durum, yorum (ziyaretID ziyaretler tablosuna, tetkikID tetkikler tablosuna referans verir))
- **bulgular** (id, tip, aciklama, kaydedilmezamani, ziyaretID (ziyaretler tablosuna referans verir))
- **ziyaretler** (id, tarihsaat, tani, sikayet, tedavinotlari, doktorid (doktorlar tablosuna referans verir), hastaid (hastalar tablosuna referans verir))
- **receteler** (id, tarih, gecерliliksuresi, ziyaretID (ziyaretler tablosuna referans verir))
- **ilaçİçerir** (receteID, ilacID, doz, adet, periyot, kullanimsekli (receteID reçeteler tablosuna, ilacID ilaçlar tablosuna referans verir))
- **duyurular** (id, baslik, metin, tarih)
- **doktorPass** (id, kid(doktorlar tablosuna referans verir), sifre)
- **yoneticiPass** (id, kid, sifre)

Fiziksel Model

a) doktorlar:

```
CREATE TABLE doktorlar (
    id INT IDENTITY(1,1) NOT NULL,
    ad NVARCHAR(50) NOT NULL,
    soyad NVARCHAR(50) NOT NULL,
    telefon NVARCHAR(20) NULL,
    diploma_no NVARCHAR(50) NULL,
    uzmanlik NVARCHAR(100) NULL,
    PRIMARY KEY CLUSTERED (id ASC));
```

b) hastalar:

```
CREATE TABLE hastalar (
    id INT IDENTITY(1,1) NOT NULL,
    tc_no NVARCHAR(11) NOT NULL,
    ad NVARCHAR(50) NOT NULL,
    soyad NVARCHAR(50) NOT NULL,
    telefon NVARCHAR(20) NULL,
    dogum_tarihi DATE NULL,
    cinsiyet INT NULL,
    adres NVARCHAR(MAX) NULL,
    kan_grubu NVARCHAR(10) NULL,
    PRIMARY KEY CLUSTERED (id ASC),
    UNIQUE NONCLUSTERED (tc_no ASC));
```


c) hasta_alerjiler:

```
CREATE TABLE hasta_alerjiler (  
    id INT IDENTITY(1,1) NOT NULL,  
    hasta_id INT NOT NULL,  
    alerji_adi NVARCHAR(100) NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_alerjiler_hastalar  
        FOREIGN KEY (hasta_id) REFERENCES hastalar (id) ON DELETE  
CASCADE);
```

d) hasta_kronik_hastaliklar:

```
CREATE TABLE hasta_kronik_hastaliklar (  
    id INT IDENTITY(1,1) NOT NULL,  
    hasta_id INT NOT NULL,  
    hastalik_adi NVARCHAR(100) NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_kronik_hastalar  
        FOREIGN KEY (hasta_id) REFERENCES hastalar (id) ON DELETE  
CASCADE);
```

e) ziyaretler:

```
CREATE TABLE ziyaretler (  
    id INT IDENTITY(1,1) NOT NULL,  
    tarihsaat DATETIME NULL,  
    sikayet NVARCHAR(MAX) NULL,  
    tani NVARCHAR(MAX) NULL,  
    tedavinotlari NVARCHAR(MAX) NULL,  
    doktorid INT NULL,  
    hastaid INT NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_ziyaretler_doktorlar  
        FOREIGN KEY (doktorid) REFERENCES doktorlar (id) ON DELETE SET  
NULL,  
    CONSTRAINT fk_ziyaretler_hastalar  
        FOREIGN KEY (hastaid) REFERENCES hastalar (id) ON DELETE CASCADE);
```

f) bulgular:

```
CREATE TABLE bulgular (  
    id INT IDENTITY(1,1) NOT NULL,  
    aciklama NVARCHAR(MAX) NULL,  
    tip INT NULL,  
    kaydedilmezamani DATETIME NULL,  
    ziyaretid INT NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_bulgular_ziyaretler  
        FOREIGN KEY (ziyaretid) REFERENCES ziyaretler (id) ON DELETE  
        CASCADE);
```

g) ilaclar:

```
CREATE TABLE ilaclar (  
    id INT IDENTITY(1,1) NOT NULL,  
    barkod NVARCHAR(50) NULL,  
    ad NVARCHAR(100) NULL,  
    etkenmadde NVARCHAR(100) NULL,  
    form NVARCHAR(50) NULL,  
    dozajbilgisi NVARCHAR(100) NULL,  
    PRIMARY KEY CLUSTERED (id ASC));
```

h) receteler:

```
CREATE TABLE receteler (  
    id INT IDENTITY(1,1) NOT NULL,  
    tarih DATE NULL,  
    gecerliliksuresi INT NULL,  
    ziyaretid INT NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_receteler_ziyaretler  
        FOREIGN KEY (ziyaretid) REFERENCES ziyaretler (id) ON DELETE  
        CASCADE);
```

i) tetkikler:

```
CREATE TABLE tetkikler (  
    id INT IDENTITY(1,1) NOT NULL,  
    ad NVARCHAR(100) NULL,  
    aciklama NVARCHAR(MAX) NULL,  
    nderkek NVARCHAR(100) NULL,  
    ndkadin NVARCHAR(100) NULL,  
    PRIMARY KEY CLUSTERED (id ASC));
```

j) istenentetkikler:

```
CREATE TABLE istenentetkikler (  
    id INT IDENTITY(1,1) NOT NULL,  
    istektarihi DATETIME NULL,  
    sonuctarihi DATETIME NULL,  
    sonuc NVARCHAR(MAX) NULL,  
    yorum NVARCHAR(MAX) NULL,  
    durum INT NULL,  
    ziyaretid INT NULL,  
    tetkikid INT NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_istenentetkikler_tetkikler  
        FOREIGN KEY (tetkikid) REFERENCES tetkikler (id),  
    CONSTRAINT fk_istenentetkikler_ziyaretler  
        FOREIGN KEY (ziyaretid) REFERENCES ziyaretler (id) ON DELETE  
    CASCADE);
```

l) recetekalemleri:

```
CREATE TABLE recetekalemleri (  
    id INT IDENTITY(1,1) NOT NULL,  
    kullanimsekli NVARCHAR(100) NULL,  
    doz NVARCHAR(50) NULL,  
    periyot NVARCHAR(50) NULL,  
    adet INT NULL,  
    receteid INT NULL,  
    ilacid INT NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_recetekalemleri_ilaclar  
        FOREIGN KEY (ilacid) REFERENCES ilaclar (id),  
    CONSTRAINT fk_recetekalemleri_receteler  
        FOREIGN KEY (receteid) REFERENCES receteler (id) ON DELETE  
    CASCADE);
```

m) duyurular:

```
CREATE TABLE duyurular (  
    id INT IDENTITY(1,1) NOT NULL,  
    baslik NVARCHAR(255) NULL,  
    metin NVARCHAR(MAX) NULL,  
    tarih DATE NULL,  
    PRIMARY KEY CLUSTERED (id ASC));
```

n) doktorpass:

```
CREATE TABLE doktorpass (  
    id INT IDENTITY(1,1) NOT NULL,  
    kid INT NOT NULL,  
    sifre NVARCHAR(100) NOT NULL,  
    PRIMARY KEY CLUSTERED (id ASC),  
    CONSTRAINT fk_doktorpass_doktorlar  
        FOREIGN KEY (kid) REFERENCES doktorlar (id) ON DELETE CASCADE);
```

o) yoneticipass:

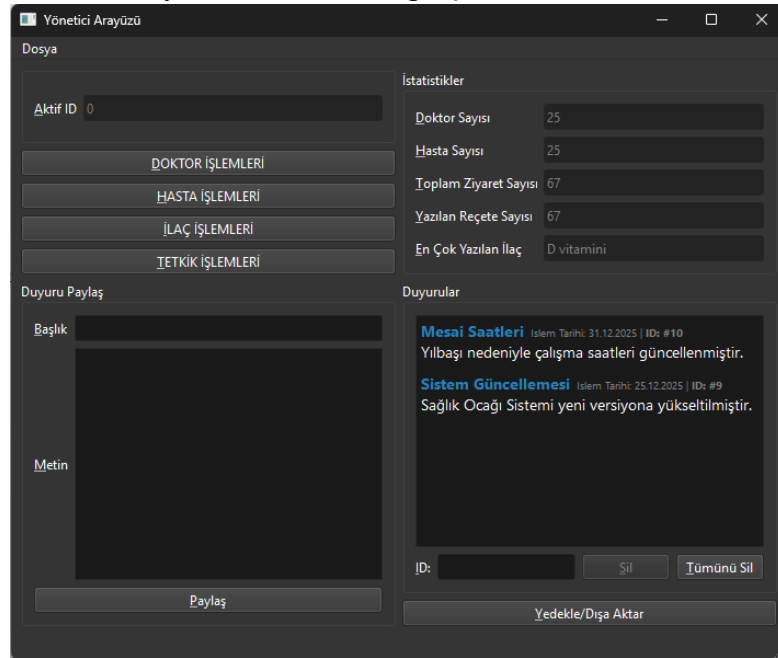
```
CREATE TABLE yoneticipass (  
    id INT IDENTITY(1,1) NOT NULL,  
    kid INT NOT NULL,  
    sifre NVARCHAR(100) NOT NULL,  
    PRIMARY KEY CLUSTERED (id ASC));
```

Ekran Görüntüleri

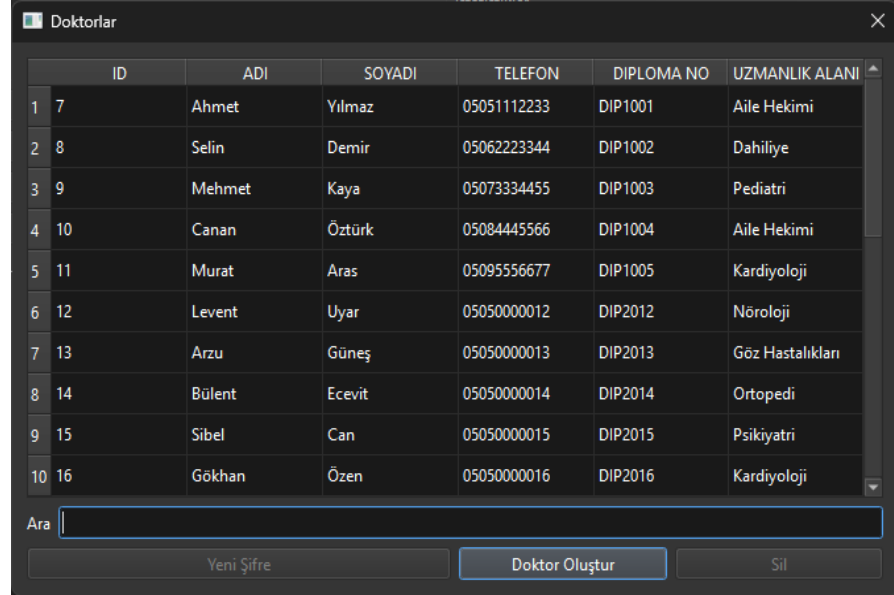
Uygulamanın kullanıcı arayüzleri, rollerine göre (Yönetici ve Doktor) ayrılmış ve her bir modül modüler bir yapıda tasarlanmıştır.

a) Yönetici Arayüzü

Yönetici paneli, sistemin idari yönetimini ve veri girişlerini kontrol eden merkezi birimdir.



1. **Doktor Listesi:** Sistemde kayıtlı olan tüm doktorların diploma no, uzmanlık ve iletişim bilgilerinin tablo üzerinde listelendiği ekrandır.

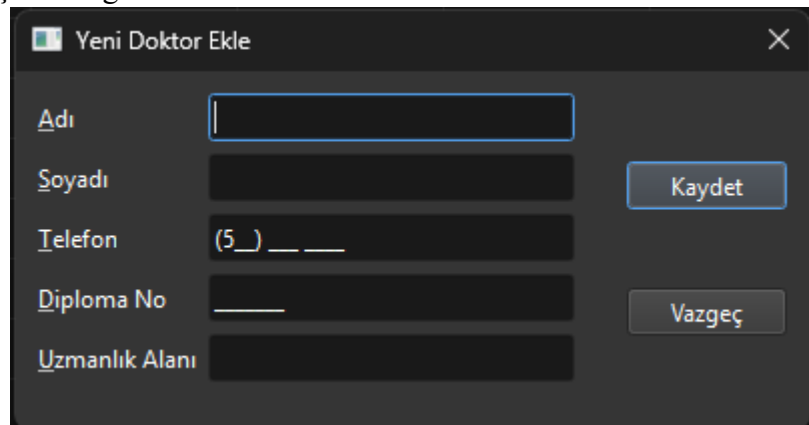


ID	ADI	SOYADI	TELEFON	DIPLOMA NO	UZMANLIK ALANI
7	Ahmet	Yılmaz	05051112233	DIP1001	Aile Hekimi
8	Selin	Demir	05062223344	DIP1002	Dahiliye
9	Mehmet	Kaya	05073334455	DIP1003	Pediyatri
10	Canan	Öztürk	05084445566	DIP1004	Aile Hekimi
11	Murat	Aras	05095556677	DIP1005	Kardiyoloji
12	Levent	Uyar	05050000012	DIP2012	Nöroloji
13	Arzu	Güneş	05050000013	DIP2013	Göz Hastalıkları
14	Bülent	Ecevit	05050000014	DIP2014	Ortopedi
15	Sibel	Can	05050000015	DIP2015	Psikiyatri
16	Gokhan	Özen	05050000016	DIP2016	Kardiyoloji

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki doktorlar tablosundan veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```
1. QSqlQuery qDoktor("SELECT * FROM doktorlar");
2. while(qDoktor.next()){
3.     auto d = tblDoktor.olustur();
4.     d->setId(qDoktor.value("id").toInt());
5.     d->setadi(qDoktor.value("ad").toString());
6.     d->setsoyadi(qDoktor.value("soyad").toString());
7.     d->settelefon(qDoktor.value("telefon").toString());
8.     d->setdiplomano(qDoktor.value("diploma_no").toString());
9.     d->setuzmanlikAlani(qDoktor.value("uzmanlik").toString());
10.    tblDoktor.ramEkle(d);
11. }
```

2. **Doktor Ekleme:** Yeni göreve başlayan doktorların sisteme tanımlandığı ve giriş şifrelerinin oluşturulduğu formdur.



Yeni Doktor Ekle

Adı

Soyadı

Telefon (5_)

Diploma No

Uzmanlık Alanı

Kaydet

Vazgeç

```

1. tblDoktor.setSqlEklemeIslemi([this](DoktorTablosu:VeriPointer d){
2.     QSqlQuery q;
3.     q.prepare("INSERT INTO doktorlar (ad, soyad, telefon, diploma_no, uzmanlik) VALUES (?, ?, ?, ?, ?)");
4.     q.addBindValue(d->adi());
5.     q.addBindValue(d->soyadi());
6.     q.addBindValue(d->telefon());
7.     q.addBindValue(d->diplomaNo());
8.     q.addBindValue(d->uzmanlikAlani());
9.     if(q.exec()){
10.        d->setId(q.lastInsertId().toInt());
11.    } else { qDebug() << "Doktor Ekleme Hatası:" << q.lastError().text(); }
12. });

```

3. **Doktor Silme:** Görevden ayrılan doktorları silme işlemini yapar. Doktor liste formunda sil butonu ile yapılır.

	ID	ADI	SOYADI	TELEFON	DIPLOMA NO	ZMANLIK ALANI
1	7	Ahmet	Yılmaz	05051112233	DIP1001	Aile Hekimi
2	8	Selin	Demir	05062223344	DIP1002	Dahiliye
3	9	Mehmet	Kaya	05073334455	DIP1003	Pediyatri
4	10	Canan	Öztürk	05084445566	DIP1004	Aile Hekimi
5	11	Murat	Aras	05095556677	DIP1005	Kardiyoloji
6	12	Levent	Uyar	05050000012	DIP2012	Nöroloji
7	13	Arzu	Güneş	05050000013	DIP2013	Göz ...
8	14	Bülent	Ecevit	05050000014	DIP2014	Ortopedi
9	15	Sibel	Can	05050000015	DIP2015	Psikiyatri

```

1. tblDoktor.setSqlSilmeIslemi([this](quint32 id){
2.     QSqlQuery q; q.prepare("DELETE FROM doktorlar WHERE id=?"); q.addBindValue(id); q.exec();
3.     q.prepare("DELETE FROM doktorpass WHERE id=?"); q.addBindValue(id); q.exec();
4. });

```

4. **Hasta Listesi:** Kayıtlı tüm hastaların genel listesidir. Arama fonksiyonu ile TC No üzerinden hızlı erişim sağlanır.

	ID	ADI	SOYADI	CINSİYET	DOĞUM TARİHİ	TC KİMLİK	TELEFON	ADRES	KAN GRUBU	ALERJİLER	İNİK HASTALIK	ZİYARETLER
1	14	Murat	Yılmaz	Kadın	10.01.1980	10000000001	05321000001	Çankaya, ...	A Rh+			32, 43, 57
2	15	Elif	Kaya	Erkek	22.03.1992	10000000002	05321000002	Kadıköy, ...	B Rh-			24, 39
3	16	Hakan	Demir	Kadın	15.07.1975	10000000003	05321000003	Nilüfer, Bursa	O Rh+			33, 51
4	17	Selin	Şahin	Erkek	05.11.1988	10000000004	05321000004	Konak, İzmir	AB Rh+			26, 44, 58, 6...
5	18	Oğuz	Çelik	Kadın	30.05.2000	10000000005	05321000005	Ortahisar, ...	A Rh-		Diyabet	28
6	19	Buse	Aydın	Erkek	12.09.1995	10000000006	05321000006	Sultanpinar, ...	B Rh+	Lateks		25, 69
7	20	Mert	Koç	Kadın	01.12.1965	10000000007	05321000007	Kocasinan, ...	O Rh-			23, 34, 41
8	21	Aslı	Kurt	Erkek	28.02.1983	10000000008	05321000008	Merkez, ...	A Rh+	Fındık		77
9	22	Emre	Özcan	Kadın	18.06.1970	10000000009	05321000009	İlkadım, ...	B Rh+		KOAH	45, 53, 76

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki hastalar ve hastanın ilişkili olduğu hasta_alerjiler ve hasta_kronik_hastalıklar tablolarından veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```
1. QSqlQuery qHasta("SELECT * FROM hastalar");
2. while(qHasta.next()){
3.     auto h = tblHasta.olustur();
4.     h->setId(qHasta.value("id").toInt());
5.     h->setTckimlik(qHasta.value("tc_no").toString());
6.     h->setadi(qHasta.value("ad").toString());
7.     h->setsoyadi(qHasta.value("soyad").toString());
8.     h->settelefon(qHasta.value("telefon").toString());
9.     h->setDogumTarihi(qHasta.value("dogum_tarihi").toDate());
10.    h->setCinsiyet((CinsiyetEnum)qHasta.value("cinsiyet").toInt());
11.    h->setAdres(qHasta.value("adres").toString());
12.    h->setKanGrubu(qHasta.value("kan_grubu").toString());
13.
14.    QStringList alerjilistesi;
15.    QSqlQuery qAlerji;
16.    qAlerji.prepare("SELECT alerji_adi FROM hasta_alerjiler WHERE hasta_id=?");
17.    qAlerji.addBindValue(h->id());
18.    if(qAlerji.exec()){
19.        while (qAlerji.next()) {
20.            alerjilistesi.append(qAlerji.value(0).toString());
21.        }
22.    }
23.
24.    h->setAlerjiler(alerjilistesi);
25.
26.    QStringList khastaliklistesi;
27.    QSqlQuery qKhastalik;
28.    qKhastalik.prepare("SELECT hastalik_adi FROM hasta_kronik_hastaliklar WHERE hasta_id=?");
29.    qKhastalik.addBindValue(h->id());
30.    if(qKhastalik.exec()){
31.        while (qKhastalik.next()) {
32.            khastaliklistesi.append(qKhastalik.value(0).toString());
33.        }
34.    }
35.    h->setKronikHastaliklar(khastaliklistesi);
36.    tblHasta.ramEkle(h);
37. }
```

5. Hasta Ekleme: Yeni hasta kayıtlarının (TC, Kan Grubu, Adres vb.) sisteme ilk girişinin yapıldığı ekrandır.

```

1. tblHasta.setSqlEklemeIslemi([this](HastaTablosu::VeriPointer h){
2.     QSqlQuery q;
3.     q.prepare("INSERT INTO hastalar (tc_no, ad, soyad, telefon, dogum_tarihi, cinsiyet, adres, kan_grubu)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)");
4.     q.addBindValue(h->tcKimlik());
5.     q.addBindValue(h->adi());
6.     q.addBindValue(h->soyadi());
7.     q.addBindValue(h->telefon());
8.     q.addBindValue(h->dogumTarihi());
9.     q.addBindValue(static_cast<int>(h->cinsiyet()));
10.    q.addBindValue(h->adres());
11.    q.addBindValue(h->kanGrubu());
12.
13.    if(q.exec()) {
14.        int yeniHastaid = q.lastInsertId().toInt();
15.        h->setId(yeniHastaid);
16.
17.        QSqlQuery qAlt;
18.        qAlt.prepare("INSERT INTO hasta_alerjiler (hasta_id, alerji_adi) VALUES (?, ?)");
19.        for(QString &alerji:h->alerjiler()){
20.            qAlt.bindValue(0, yeniHastaid);
21.            qAlt.bindValue(1, alerji);
22.            qAlt.exec();
23.        }
24.        qAlt.prepare("INSERT INTO hasta_kronik_hastaliklar (hasta_id, hastalik_adi) VALUES (?, ?)");
25.        for(QString &hastalik:h->kronikHastaliklar()){
26.            qAlt.bindValue(0, yeniHastaid);
27.            qAlt.bindValue(1, hastalik);
28.            qAlt.exec();
29.        }
30.    }
31.    else qDebug() << "Hasta Ekleme Hatası:" << q.lastError().text();
32. });

```

6. Hasta Düzenleme: Bilgileri değişen hastaların sistemdeki bilgilerinin düzenlenmesini sağlar.

The screenshot shows a Qt-style window titled "Hasta Düzenle" with a close button (X) in the top right corner. The window is divided into two main sections. The left section contains input fields for patient details: "Adı" (Murat), "Soyadı" (Yılmaz), "Cinsiyet" (KADIN), "Doğum Tarihi" (10.01.1980), "Kimlik No" (10000000001), "Telefon No" ((532) 100 00 01), and "Adres" (Çankaya, Ankara). The right section has a "Kan Grubu" dropdown (A+), an "Alerjiler" section with a large empty text area, and a "Kronik Hastalıklar" section with another large empty text area. Below the "Alerjiler" section are "Ekle" and "Sil" buttons. At the bottom of the window are four buttons: "Kaydet", "Vazgeç", "Ekle", and "Sil".


```

1. tblHasta.setSqlGuncellemeIslemi([(HastaTablosu::VeriPointer h){
2.     QSqlQuery q;
3.     q.prepare("UPDATE hastalar SET tc_no=?, ad=?, soyad=?, telefon=?, dogum_tarihi=?, cinsiyet=?, adres=?,
kan_grubu=? WHERE id=?");
4.     q.addBindValue(h->tcKimlik());
5.     q.addBindValue(h->adi());
6.     q.addBindValue(h->soyadi());
7.     q.addBindValue(h->telefon());
8.     q.addBindValue(h->dogumTarihi());
9.     q.addBindValue(static_cast<int>(h->cinsiyet()));
10.    q.addBindValue(h->adres());
11.    q.addBindValue(h->kanGrubu());
12.    q.addBindValue(h->id());
13.    if(!q.exec()) {
14.        qDebug() << "Hasta Güncelleme Hatası: " << q.lastError().text();
15.        return;
16.    }
17.    QSqlQuery qSil;
18.    qSil.prepare("DELETE FROM hasta_alerjiler WHERE hasta_id=?");
19.    qSil.addBindValue(h->id());
20.    qSil.exec();
21.    qSil.prepare("DELETE FROM hasta_kronik_hastaliklar WHERE hasta_id=?");
22.    qSil.addBindValue(h->id());
23.    qSil.exec();
24.    QSqlQuery qEkle;
25.    qEkle.prepare("INSERT INTO hasta_alerjiler (hasta_id, alerji_adi) VALUES (?,?)");
26.    for(QString &alerji:h->alerjiler()){
27.        qEkle.bindValue(0,h->id());
28.        qEkle.bindValue(1,alerji);
29.        qEkle.exec();
30.    }
31.    qEkle.prepare("INSERT INTO hasta_kronik_hastaliklar (hasta_id, hastalik_adi) VALUES (?,?)");
32.    for(QString &hastalik:h->kronikHastaliklar()){
33.        qEkle.bindValue(0,h->id());
34.        qEkle.bindValue(1,hastalik);
35.        qEkle.exec();
36.    }
37. });

```

7. **Hasta Silme:** Hatalı veya mükerrer kayıtların sistemden temizlenmesini sağlar. Hasta liste formunda sil butonu ile yapılır.

ID	ADI	SOYADI	CINSİYET	DOĞUM TARİHİ	TC KİMLİK	TELEFON	ADRES	KAN GRUBU	ALERJİLER	KRONİK HASTALIKI	ZİYARETLER
1	14	Murat	Yılmaz	Kadın	10.01.1980	10000000001	05321000001	Çankaya, ...	A Rh+		32, 43, 57
2	15	Elif	Kaya	Erkek	22.03.1992	10000000002	05321000002	Kadıköy, ...	B Rh-		24, 39
3	16	Hakan	Demir	Kadın	15.07.1975	10000000003	05321000003	Nilüfer, Bursa	O Rh+		33, 51
4	17	Selin	Şahin	Erkek	05.11.1988	10000000004	05321000004	Konak, İzmir	AB Rh+		26, 44, 58, 61, ...
5	18	Oğuz	Çelik	Kadın	30.05.2000	10000000005	05321000005	Ortahisar, ...	A Rh-		Diyabet
6	19	Buse	Aydın	Erkek	12.09.1995	10000000006	05321000006	Sultanpınar, ...	B Rh+	Lateks	25, 69
7	20	Mert	Koç	Kadın	01.12.1965	10000000007	05321000007	Kocasinan, ...	O Rh-		23, 34, 41
8	21	Aslı	Kurt	Erkek	28.02.1983	10000000008	05321000008	Merkez, ...	A Rh+	Fındık	77
9	22	Emre	Özcan	Kadın	18.06.1970	10000000009	05321000009	İlkadım, ...	B Rh+		KOAH
...	...	...	...	...	...	...	...	...	...	...	...

```

1. tblHasta.setSqlSilmeIslemi([this](quint32 id){
2.     QSqlQuery q;
3.     q.prepare("DELETE FROM hasta_alerjiler WHERE hasta_id=?"); q.addBindValue(id); q.exec();
4.     q.prepare("DELETE FROM hasta_kronik_hastaliklar WHERE hasta_id=?"); q.addBindValue(id); q.exec();
5.     q.prepare("DELETE FROM hastalar WHERE id=?"); q.addBindValue(id); q.exec();
6.     auto ziyaret=VERITABANI::vt().ziyaretler().bul([&id](ZiyaretTablosu::VeriPointer d){
7.         return d->hastaId()==id;
8.     });
9.     for(auto &i:ziyaret){
10.         VERITABANI::vt().ziyaretler().IdyeGoreSil(i->id());
11.     }
12. });

```

8. **İlaç Yönetimi:** Reçeteleme sürecinde kullanılacak olan ilaçların barkod ve etken madde bilgilerinin yönetildiği modüldür. (İlaç Ekleme/Silme/Düzenleme işlemleri önceki işlemleri ile aynı şekilde yapılmaktadır.)

İlaçlar

</

```
1. tblIlac.setSqlEklemeIslemi([this](IlacTablosu::VeriPointer i){
2.     QSqlQuery q; q.prepare("INSERT INTO ilacler (barkod,ad,etkenmadde,form,dozajbilgisi) VALUES (?,?,,?,?)");
3.     q.addBindValue(i->barkod());
4.     q.addBindValue(i->ad());
5.     q.addBindValue(i->etkenMadde());
6.     q.addBindValue(i->form());
7.     q.addBindValue(i->dozajBilgisi());
8.
9.     if(q.exec()) i->setId(q.lastInsertId().toInt());
10.    else qDebug() << "İlaç Ekleme Hatası: ";
11. });
12.
13. tblIlac.setSqlSilmeIslemi([this](quint32 id){
14.     QSqlQuery q; q.prepare("DELETE FROM ilacler WHERE id=?"); q.addBindValue(id); q.exec();
15. });
16.
17. tblIlac.setSqlGuncellemeIslemi([this](IlacTablosu::VeriPointer i){
18.     QSqlQuery q;
19.     q.prepare("UPDATE ilacler SET barkod=?, ad=?, etkenmadde=?, form=?, dozajbilgisi=? WHERE id=?");
20.     q.addBindValue(i->barkod());
21.     q.addBindValue(i->ad());
22.     q.addBindValue(i->etkenMadde());
23.     q.addBindValue(i->form());
24.     q.addBindValue(i->dozajBilgisi());
25.     q.addBindValue(i->id());
26.     if(!q.exec()) qDebug() << "İlaç Güncelleme Hatası:" << q.lastError().text();
27. });
```

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki ilaçlar tablosundan veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```

1. QSqlQuery qIlac("SELECT * FROM ilaclar");
2. while (qIlac.next()) {
3.     auto i = tblIlac.olustur();
4.     i->setId(qIlac.value("id").toInt());
5.     i->setBarkod(qIlac.value("barkod").toString());
6.     i->setAd(qIlac.value("ad").toString());
7.     i->setEtkenMadde(qIlac.value("etkenmadde").toString());
8.     i->setForm(qIlac.value("form").toString());
9.     i->setDozajBilgisi(qIlac.value("dozajbilgisi").toString());
10.
11.     tblIlac.ramEkle(i);
12. }

```

9. **Tetkik Tanımlama:** Laboratuvar biriminde yapılabilecek tetkiklerin ve referans değerlerinin kütüphaneye eklendiği kısımdır. (Tetkik Ekleme/Silme/Düzenleme işlemleri önceki işlemleri ile aynı şekilde yapılmaktadır.)

ID	TETKİK ADI	ACIKLAMA	MAL DEGERLER EF	MAL DEGERLER K/
7	Glikoz (Açlık)	Kan şekeri ...	70 - 100 mg/dL	70 - 100 mg/dL
8	B12 Vitamini	Kobalamin ...	200 - 900 pg/mL	200 - 900 pg/mL
9	TSH	Tiroid uyarıcı ...	0.4 - 4.0 mIU/L	0.4 - 4.0 mIU/L
10	Alanin ...	Karaciğer ...	0 - 41 U/L	0 - 33 U/L
11	Aspartat ...	Karaciğer ve ka...	0 - 40 U/L	0 - 32 U/L
12	Üre	Böbrek fonksiy...	10 - 50 mg/dL	10 - 50 mg/dL
13	Kreatinin	Böbrek süzme ...	0.7 - 1.2 mg/dL	0.5 - 1.0 mg/dL
14	Tam İdrar Tahlili	İdrar yolu ...	Negatif	Negatif
15	Sodyum (Na)	Elektrolit denge...	135 - 145 mmol/L	135 - 145 mmol/L
16	Potasyum (K)	Kalp ve kas ...	3.5 - 5.1 mmol/L	3.5 - 5.1 mmol/L
17	Kalsiyum (Ca)	Kemik ve miner...	8.6 - 10.2 mg/dL	8.6 - 10.2 mg/dL
18	HDL Kolesterol	İyi huylu ...	> 40 mg/dL	> 50 mg/dL
19	LDL Kolesterol	Kötü huylu ...	< 130 mg/dL	< 130 mg/dL

```

1. tblTetkik.setSqlEklemeIslemi([this](TetkikTablosu::VeriPointer t){
2.     QSqlQuery q; q.prepare("INSERT INTO tetkikler (ad,aciklama,nderkek,ndkadin) VALUES (?, ?, ?, ?)");
3.     q.addBindValue(t->ad());
4.     q.addBindValue(t->aciklama());
5.     q.addBindValue(t->normalDegerlerErkek());
6.     q.addBindValue(t->normalDegerlerKadin());
7.
8.     if(q.exec()) t->setId(q.lastInsertId().toInt());
9.     else qDebug() << "Tetkik Ekleme Hatası: ";
10. });
11.
12. tblTetkik.setSqlSilmeIslemi([this](quint32 id){
13.     QSqlQuery q; q.prepare("DELETE FROM tetkikler WHERE id=?"); q.addBindValue(id); q.exec();
14. });
15.
16. tblTetkik.setSqlGuncellemeIslemi([this](TetkikTablosu::VeriPointer t){
17.     QSqlQuery q;
18.     q.prepare("UPDATE tetkikler SET ad=?, aciklama=?, nderkek=?, ndkadin=? WHERE id=?");
19.     q.addBindValue(t->ad());
20.     q.addBindValue(t->aciklama());
21.     q.addBindValue(t->normalDegerlerErkek());
22.     q.addBindValue(t->normalDegerlerKadin());
23.     q.addBindValue(t->id());
24.
25.     if(!q.exec()) qDebug() << "Tetkik Güncelleme Hatası: " << q.lastError().text();
26. });

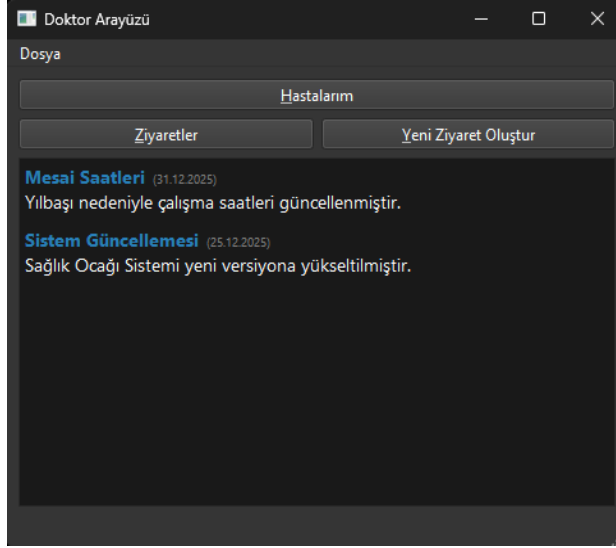
```

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki tetkikler tablosundan veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```
1.  QSqlQuery qTetikik("SELECT * FROM tetkikler");
2.  while (qTetikik.next()) {
3.      auto t=tblTetikik.olustur();
4.      t->setId(qTetikik.value("id").toInt());
5.      t->setAd(qTetikik.value("ad").toString());
6.      t->setAciklama(qTetikik.value("aciklama").toString());
7.      t->setNormalDegerlerErkek(qTetikik.value("nderkek").toString());
8.      t->setNormalDegerlerKadin(qTetikik.value("ndkadin").toString());
9.
10.     tblTetikik.ramEkle(t);
11. }
```

b) Doktor Arayüzü

Doktor arayüzü, hastaların tıbbi süreçlerinin takip edildiği, tanı ve tedavi işlemlerinin dijital ortama aktarıldığı ana çalışma alanıdır.



1. **Hasta Listesi:** Sisteme kayıtlı tüm hastaların veya doktora atanmış hastaların listelendiği ekrandır. TC Kimlik No ile hızlı arama yapılarak ilgili hastanın dosyasına erişilir.

ID	ADI	SOYADI	CINSİYET	DOĞUM TARİHİ	TC KİMLİK	TELEFON	ADRES	KAN GRUBU	ALERJİLER	ONİK HASTALIKLI
1	14	Murat	Yılmaz	Kadın	10.01.1980	10000000001	05321000001	Çankaya, Ankara	A Rh+	
2	15	Elif	Kaya	Erkek	22.03.1992	10000000002	05321000002	Kadıköy, İstanbul	B Rh+	
3	16	Hakan	Demir	Kadın	15.07.1975	10000000003	05321000003	Nilüfer, Bursa	O Rh+	
4	17	Selin	Şahin	Erkek	05.11.1988	10000000004	05321000004	Konak, İzmir	AB Rh+	
5	18	Oğuz	Çelik	Kadın	30.05.2000	10000000005	05321000005	Ortahisar, ...	A Rh-	Diyabet
6	19	Buse	Aydın	Erkek	12.09.1995	10000000006	05321000006	Sultanpınar, ...	B Rh+	Lateks
7	20	Mert	Koç	Kadın	01.12.1965	10000000007	05321000007	Kocasinan, ...	O Rh-	
8	21	Aslı	Kurt	Erkek	28.02.1983	10000000008	05321000008	Merkez, Amasya	A Rh+	Fındık
9	22	Emre	Özcan	Kadın	18.06.1970	10000000009	05321000009	İlkadım, Samsun	B Rh+	KOAH
10	23	Gizem	Yıldız	Erkek	04.04.1998	10000000010	05321000010	Tepebaşı, ...	AB Rh-	
11	24	Deniz	Arslan	Kadın	10.10.1990	10000000011	05321000011	Seyhan, Adana	O Rh+	
12	25	İrem	Bulut	Erkek	20.01.2005	10000000012	05321000012	İsmet Konağı, ...	A Rh-	An Zehri

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki hastalar ve hastanın ilişkili olduğu hasta_alerjiler ve hasta_kronik_hastalıklar tablolarından veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```
1. QSqlQuery qHasta("SELECT * FROM hastalar");
2. while(qHasta.next()){
3.     auto h = tblHasta.olustur();
4.     h->setId(qHasta.value("id").toInt());
5.     h->setTckimlik(qHasta.value("tc_no").toString());
6.     h->setadi(qHasta.value("ad").toString());
7.     h->setsoyadi(qHasta.value("soyad").toString());
8.     h->settelefon(qHasta.value("telefon").toString());
9.     h->setDogumTarihi(qHasta.value("dogum_tarihi").toDate());
10.    h->setCinsiyet(((CinsiyetEnum)qHasta.value("cinsiyet").toInt());
11.    h->setAdres(qHasta.value("adres").toString());
12.    h->setKanGrubu(qHasta.value("kan_grubu").toString());
13.
14.    QStringList alerjilistesi;
15.    QSqlQuery qAlerji;
16.    qAlerji.prepare("SELECT alerji_adi FROM hasta_alerjiler WHERE hasta_id=?");
17.    qAlerji.addBindValue(h->id());
18.    if(qAlerji.exec()){
19.        while (qAlerji.next()) {
20.            alerjilistesi.append(qAlerji.value(0).toString());
21.        }
22.    }
23.
24.    h->setAlerjiler(alerjilistesi);
25.
26.    QStringList khastalikListesi;
27.    QSqlQuery qKhastalik;
28.    qKhastalik.prepare("SELECT hastalik_adi FROM hasta_kronik_hastaliklar WHERE hasta_id=?");
29.    qKhastalik.addBindValue(h->id());
30.    if(qKhastalik.exec()){
31.        while (qKhastalik.next()) {
32.            khastalikListesi.append(qKhastalik.value(0).toString());
33.        }
34.    }
35.    h->setKronikHastaliklar(khastalikListesi);
36.    tblHasta.ramEkle(h);
37. }
```

2. **Ziyaret (Muayene) Listesi:** Seçili hastanın geçmişten günümüze tüm hastane başvuru kayıtlarını kronolojik olarak listeler. Bu sayede hastanın tedavi süreci geriye dönük izlenebilir.

Ziyaretler						
ID	Ziyaret Tarihi	Şikayet	Tanı	Tedavi Notları	Doktor	Hasta
1 32	Sun Dec 7 ...	Halsizlik ve ekle...	B12 Vitamin ...	Kan tahlili isten...	Mabel Matiz	Murat Yılmaz
2 43	Sat Dec 6 16:15:...	Öksürük ve nef...	Akut Bronşit	Antibiyotik ...	Selin Demir	Murat Yılmaz
3 57	Mon Dec 1 ...	Ayak bileğinde ...	Yumuşak Doku ...	Soğuk uygulama...	Mabel Matiz	Murat Yılmaz

Ara

Reçeteler

Reçete Oluştur

İstenen Tetkikler

Tetkik İste

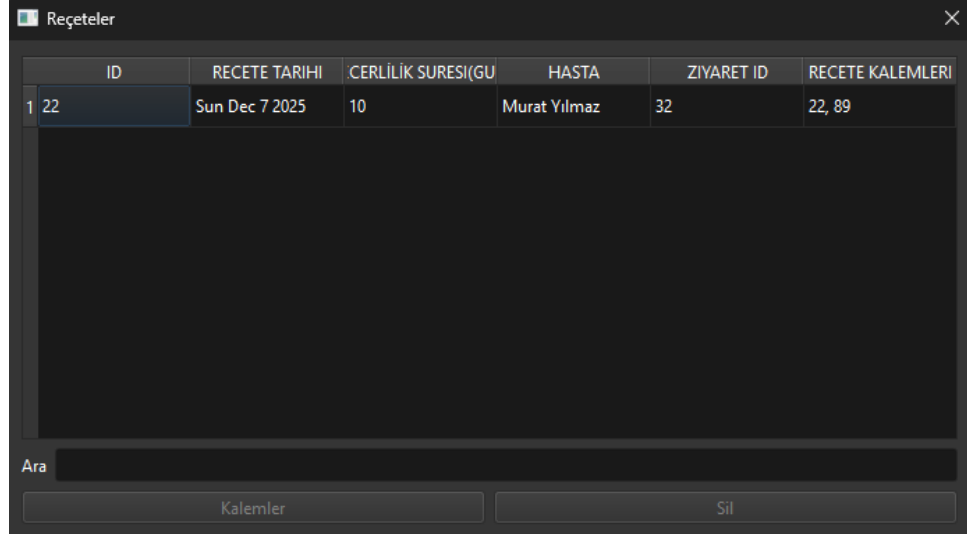
Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki ziyaretler tablosundan veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```
1. QSqlQuery qZiyaret("SELECT * FROM ziyaretler");
2. while(qZiyaret.next()){
3.     auto z = tblZiyaret.olustur();
4.     z->setId(qZiyaret.value("id").toInt());
5.     z->setTarihsaat(qZiyaret.value("tarihsaat").toDate());
6.     z->setSikayet(qZiyaret.value("sikayet").toString());
7.     z->setTani(qZiyaret.value("tani").toString());
8.     z->setTedavinotlari(qZiyaret.value("tedavinotlari").toString());
9.     z->setDoktorId(qZiyaret.value("doktorid").toInt());
10.    z->setHastaId(qZiyaret.value("hastaId").toInt());
11.
12.    tblZiyaret.ramEkle(z);
13. }
```

3. **Ziyaret (Muayene) Oluşturma:** Yeni bir muayene kaydı açarak hastanın güncel şikayet, bulgu ve tanıların veritabanına işlendiği formdur.

```
1.    tblZiyaret.setSqlEklemeIslemi([this](ZiyaretTablosu:VeriPointer z){
2.        QSqlQuery q; q.prepare("INSERT INTO ziyaretler (tarihsaat,sikayet,tani,tedavinotlari,doktorid,hastaId)
VALUES (?, ?, ?, ?, ?, ?)");
3.        q.addBindValue(z->tarihsaat());
4.        q.addBindValue(z->sikayet());
5.        q.addBindValue(z->tani());
6.        q.addBindValue(z->tedavinotlari());
7.        q.addBindValue(z->doktorid());
8.        q.addBindValue(z->hastaId());
9.
10.       if(q.exec()) z->setId(q.lastInsertId().toInt());
11.       else qDebug() << "Ziyaret Ekleme Hatası: ";
12.    });
```

4. **Reçete Listesi:** Belirli bir muayene (ziyaret) kapsamında hastaya yazılmış olan tüm reçetelerin görüntüldüğü bölümdür.

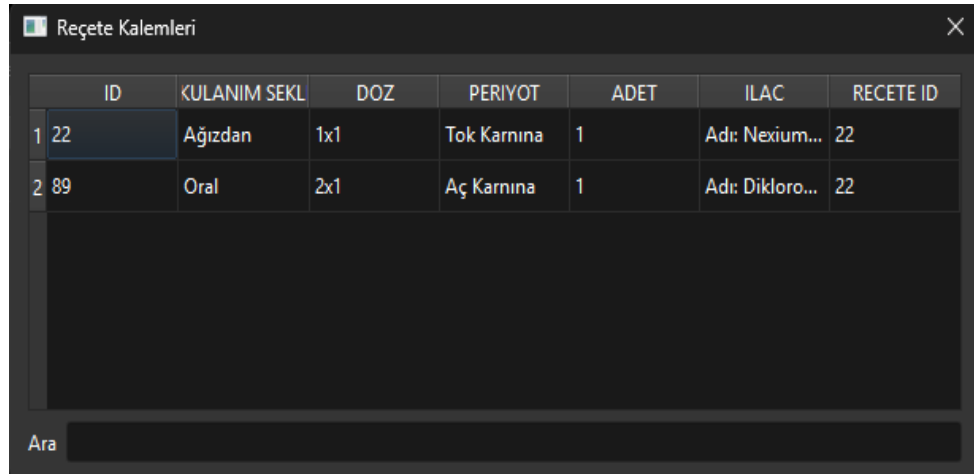


ID	RECETE TARİHİ	GERLİLİK SÜRESİ(GU)	HASTA	ZİYARET ID	RECETE KALEMLERİ
1 22	Sun Dec 7 2025	10	Murat Yılmaz	32	22, 89

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki reçeteler tablosundan veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```
1. QSqlQuery qRecete("SELECT * FROM receteler");
2. while (qRecete.next()) {
3.     auto r = tblRecete.olustur();
4.     r->setId(qRecete.value("id").toInt());
5.     r->setTarih(qRecete.value("tarih").toDate());
6.     r->setGecerlilikSuresi(qRecete.value("gecerliliksuresi").toInt());
7.     r->setZiyaretId(qRecete.value("ziyaretid").toInt());
8.
9.     tblRecete.ramEkle(r);
10. }
11.
```

5. **Reçete Kalemleri:** Reçete kalemleri modülü, ana reçete kaydı ile ilaç kütüphanesi arasındaki bağlantıyı kuran ve tedavi sürecindeki ilaç kullanım talimatlarını saklayan detay birimidir.



ID	KULANIM SEKL	DOZ	PERİYOT	ADET	İLAC	RECETE ID
1 22	Ağızdan	1x1	Tok Karnına	1	Adı: Nexium...	22
2 89	Oral	2x1	Aç Karnına	1	Adı: Dikloro...	22

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki recetekalemleri tablosundan veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```
1. QSqlQuery qReceteKalemi("SELECT * FROM recetekalemleri");
2. while (qReceteKalemi.next()) {
3.     auto r = tblReceteKalemi.olustur();
4.     r->setId(qReceteKalemi.value("id").toInt());
5.     r->setKullanimSekli(qReceteKalemi.value("kullanimsekli").toString());
6.     r->setDoz(qReceteKalemi.value("doz").toString());
7.     r->setPeriyot(qReceteKalemi.value("periyot").toString());
8.     r->setAdet(qReceteKalemi.value("adet").toInt());
9.     r->setReceteId(qReceteKalemi.value("receteid").toInt());
10.    r->setIlacID(qReceteKalemi.value("ilacid").toInt());
11.
12.    tblReceteKalemi.ramEkle(r);
13. }
```

6. **Reçete Oluştur:** Tanı ile uyumlu ilaçların, dozaj ve kullanım periyotlarıyla birlikte sisteme kaydedilerek hastaya atanmasını sağlar.

```
1. tblRecete.setSqlEklemeIslemi([this](ReceteTablosu::VeriPointer r){
2.     QSqlQuery q; q.prepare("INSERT INTO receteler (tarih,gecerliliksuresi,ziyaretid) VALUES (?,?,:)");
3.     q.addBindValue(r->tarih());
4.     q.addBindValue(r->gecerlilikSuresi());
5.     q.addBindValue(r->ziyaretid());
6.
7.     if(q.exec()) r->setId(q.lastInsertId().toInt());
8.     else qDebug() << "Reçete Ekleme Hatası: ";
9. });
```


7. **Reçete Silme:** Hatalı girilen veya iptal edilmesi gereken reçetelerin veritabanından güvenli bir şekilde kaldırılmasını sağlar (Veritabanında ON DELETE CASCADE yapısı ile ilişkilidir).

ID	REÇETE TARİHİ	RLİLİK SÜRESİ(ı)	HASTA	ZİYARET ID	CETE KALEMLİ
1	Sun Dec 7 ...	10	Murat Yılmaz	32	22, 89

Ara

Kalemler Sil

```
1. tblRecete.setSqlSilmeIslemi([this](quint32 id){
2.     QSqlQuery q; q.prepare("DELETE FROM receteler WHERE id=?"); q.addBindValue(id); q.exec();
3.     auto kalemler=VERITABANI::vt().recetekalemleri().bul([&id](ReceteKalemiTablosu::VeriPointer d){
4.         return d->receteid()==id;
5.     });
6.     for(auto kalem:kalemler){
7.         VERITABANI::vt().recetekalemleri().IdyeGoreSil(kalem->id());
8.     }
9. });
```

8. **İstenen Tetkik Listesi:** Hastadan istenmiş olan laboratuvar ve görüntüleme testlerinin durumlarını (Bekliyor/Tamamlandı) ve sonuçlarını gösteren tablodur.

ID	İSTEK TARİHİ	SONUC TARİHİ	SONUC	YORUM	DURUM	HASTA	ZİYARET ID	TETKİK
1	Thu Dec 25 02:03:...	Thu Dec 25 05:03:...	Normal Değer	Klinik olarak anla...	İSTENDİ	Murat Yılmaz	32	ID:52 Total Protein

Ara

Düzenle Sil

Verilen sorgu kodu uygulama ilk açılışında **ilkyükleme** fonksiyonu altındaki istenentetkikler tablosundan veri çekme aşamasıdır. **ilkyükleme** fonksiyonu tüm tablolardaki verileri cihaz belleğine alır işlemler bellek ve sql üzerinden eş zamanlı ilerler.

```

1.  QSqlQuery qIstenenTetkik("SELECT * FROM istenentetkikler");
2.  while(qIstenenTetkik.next()){
3.      auto i = tblIstenenTetkik.olustur();
4.      i->setId(qIstenenTetkik.value("id").toInt());
5.      i->setIstekTarihi(qIstenenTetkik.value("istektarihi").toDateTarih());
6.      i->setSonucTarihi(qIstenenTetkik.value("sonuctarihi").toDateTarih());
7.      i->setSonuc(qIstenenTetkik.value("sonuc").toString());
8.      i->setYorum(qIstenenTetkik.value("yorum").toString());
9.      i->setZiyaretId(qIstenenTetkik.value("ziyaretid").toInt());
10.     i->setTetkikId(qIstenenTetkik.value("tetkikid").toInt());
11.
12.     tblIstenenTetkik.ramEkle(i);
13. }

```

9. **İstenen Tetkik Düzenleme:** Laboratuvardan gelen sonuçların veya doktorun tetkikle ilgili eklemek istediği yorumların güncellendiği ekrandır.

```

1.  tblIstenenTetkik.setSqlGuncellemeIslemi([(IstenenTetkikTablosu::VeriPointer i){
2.      QSqlQuery q;
3.      q.prepare("UPDATE istenentetkikler SET istektarihi=?, sonuctarihi=?, sonuc=?, yorum=?, durum=?,
ziyaretid=?, tetkikid=? WHERE id=?");
4.      q.addBindValue(i->istekTarihi());
5.      q.addBindValue(i->sonucTarihi());
6.      q.addBindValue(i->sonuc());
7.      q.addBindValue(i->yorum());
8.      q.addBindValue(static_cast<int>(i->durum()));
9.      q.addBindValue(i->ziyaretid());
10.     q.addBindValue(i->tetkikid());
11.     q.addBindValue(i->id());
12.     if(!q.exec()) qDebug() << "Tetkik Sonuç Güncelleme Hatası:" << q.lastError().text();
13. });

```

10. **İstenen Tetkik Silme:** Yanlış istenen veya vazgeçilen tetkik istemlerinin sistemden temizlenmesi işlemidir.

```
1. tblIstenenTetkik.setSqlSilmeIslemi([this](quint32 id){
2.     QSqlQuery q; q.prepare("DELETE FROM istenentetkikler WHERE id=?"); q.addBindValue(id); q.exec();
3. });
```

11. **Tetkik İste:** Sistemde tanımlı olan tetkik kütüphanesinden (Hemogram, Glikoz vb.) seçim yapılarak hastaya yeni laboratuvar görevleri atanmasını sağlar.

```
1. tblIstenenTetkik.setSqlEklemeIslemi([this](IstenenTetkikTablosu::VeriPointer i){
2.     QSqlQuery q; q.prepare("INSERT INTO istenentetkikler
3.     (istektarihi,sonuctarihi,sonuc,yorum,durum,ziyaretid,tetkikid) VALUES (?,?,?,?,?,?,?)");
4.     q.addBindValue(i->istekTarihi());
5.     q.addBindValue(i->sonucTarihi());
6.     q.addBindValue(i->sonuc());
7.     q.addBindValue(i->yorum());
8.     q.addBindValue(static_cast<int>(i->durum()));
9.     q.addBindValue(i->ziyaretid());
10.    q.addBindValue(i->tetkikid());
11.    if(q.exec()) i->setId(q.lastInsertId().toInt());
12.    else qDebug() << "İstenen Tetkik Ekleme Hatası: " << q.lastError().text();
13. });
```